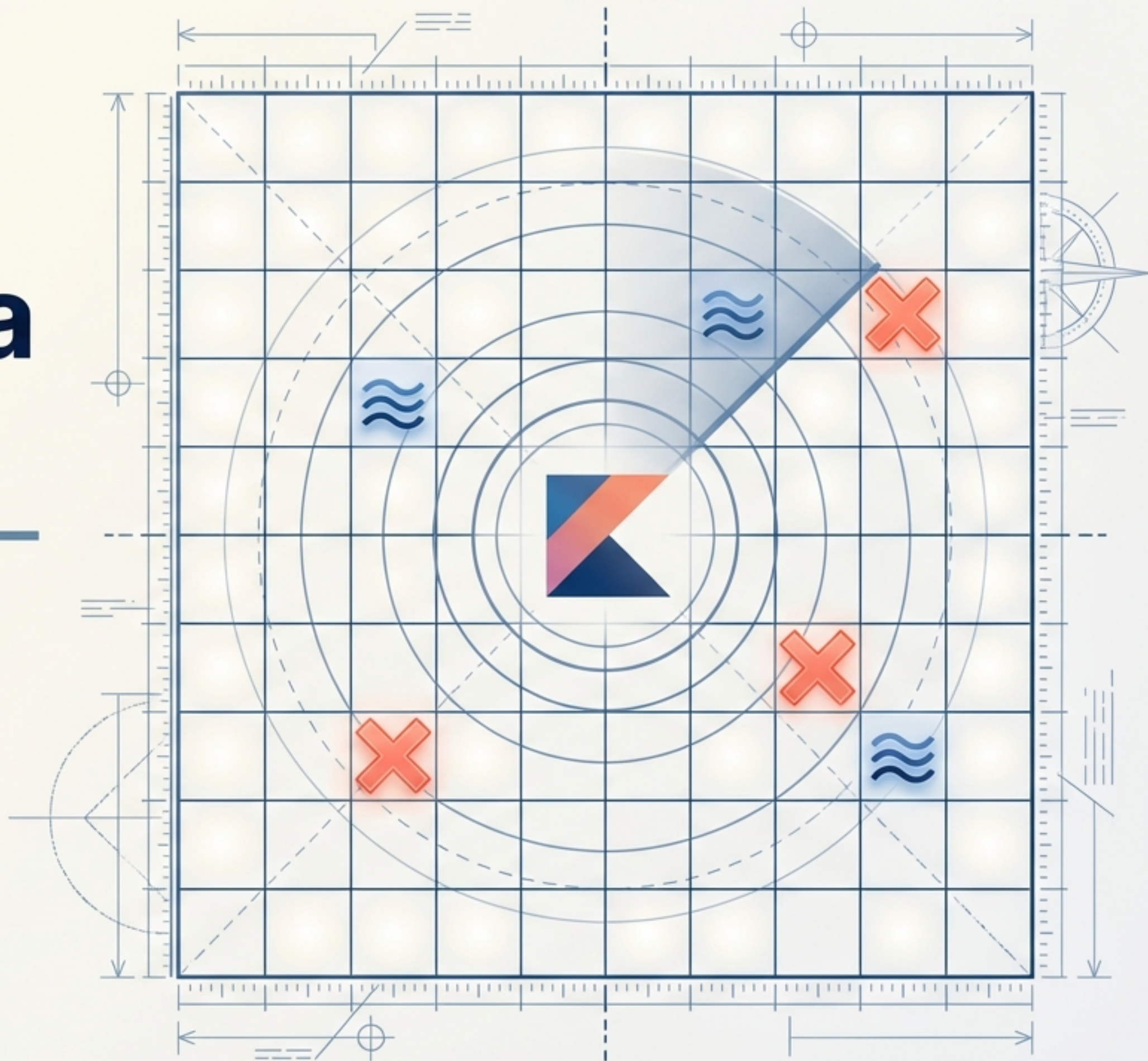


Hundir la Flota con Kotlin

Dominando Enums, Objects y MutableLists en proyectos reales.



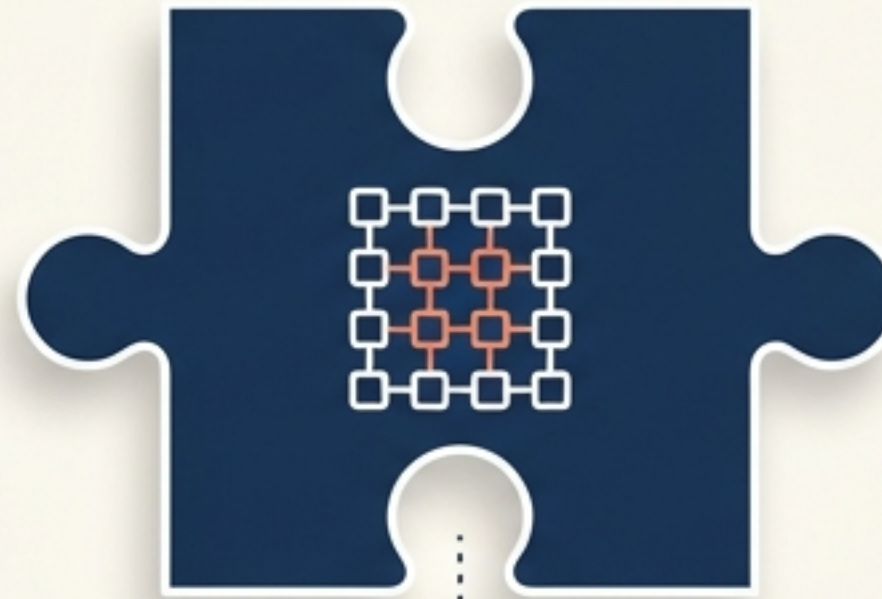
Más allá de las variables básicas

Crear un juego completo requiere resolver tres grandes desafíos estructurales:



1. Las Piezas

¿Cómo definimos los barcos y sus tamaños sin cometer errores de tipeo?



2. El Mapa

¿Cómo construimos una cuadrícula 2D (10x10) que pueda cambiar durante el juego?



3. El Estado

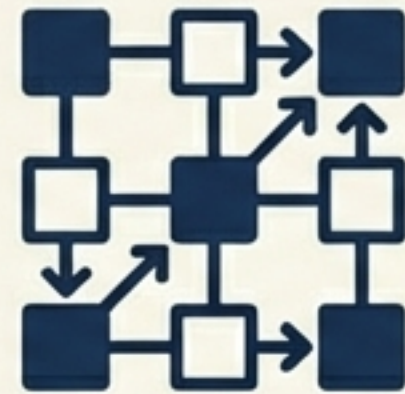
¿Dónde guardamos la información para que todo el programa pueda acceder a ella de forma segura?

Tres herramientas para organizar el caos



`enum class`

Para definir catálogos fijos de datos (los barcos y casillas).



`MutableList`

Para crear estructuras de datos dinámicas (el océano).



`object`

Para proteger y centralizar el estado global del juego (el administrador).

El peligro de usar texto y números sueltos

Fragile Code

```
val barco1 = "Portaaviones"  
val barco2 = "portaaviones" // ¡Error tipográfico!  
  
val longitudBarco = 5 // ¿De qué barco es este tamaño?
```

Usar Strings o Ints sueltos para datos fijos causa errores lógicos difíciles de rastrear.

Necesitamos una forma de vincular el nombre del barco con su tamaño de forma indestructible.

Modelando la flota con Enum Classes

El Enum no es solo una etiqueta, almacena datos reales permanentemente.

```
enum class TipoBarco(val longitud: Int) {  
    PORTAAVIONES(5), ACORAZADO(4),  
    CRUCERO(3), SUBMARINO(3), DESTRUCTOR(2)  
}
```

Seguridad absoluta: El compilador de Kotlin sabe exactamente qué barcos existen y cuánto miden.

Empaquetando lógica visual en una sola estructura

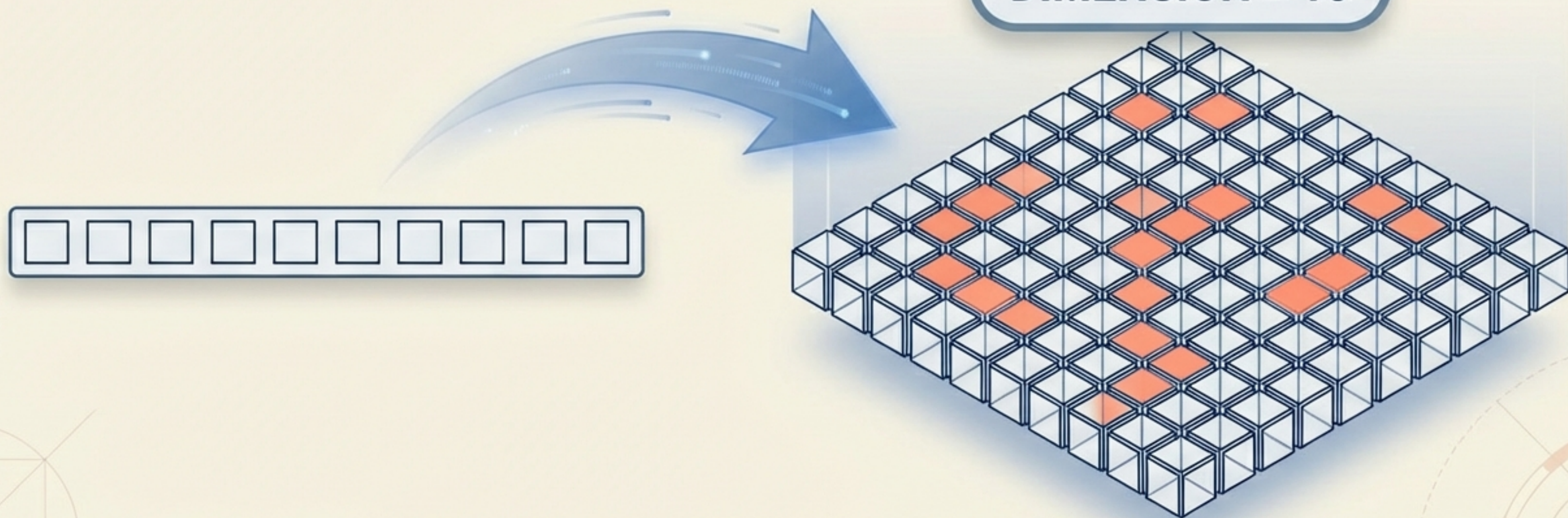
```
enum class EstadoCasilla(val simbolo: String,  
    r: Int, g: Int, b: Int) {  
    AGUA("~", 0, 180, 255),  
    BARCO("B", 160, 32, 240),  
    TOCADO("X", 255, 0, 0),  
    FALLO("o", 180, 180, 180);  
}
```



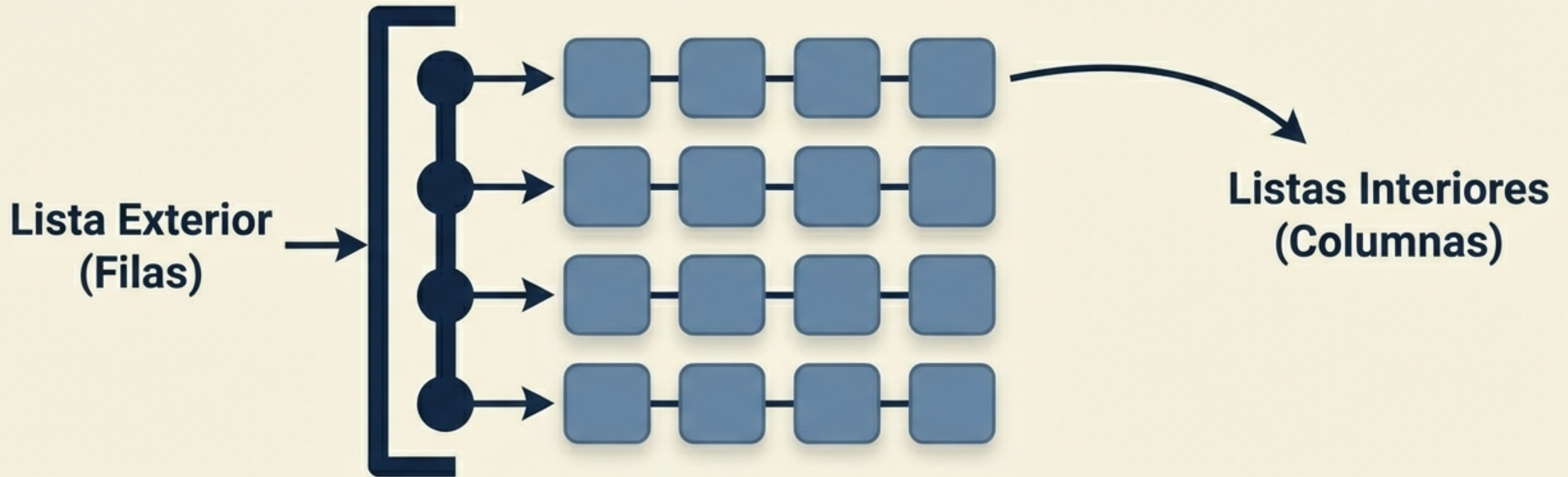
Centralizamos el símbolo y su color exacto en un solo lugar. Si queremos cambiar el aspecto visual, solo editamos este Enum.

Cómo representar un mapa bidimensional

Nuestro océano necesita ser una cuadrícula cambiante. El agua se convierte en barcos, y los barcos se convierten en impactos.



Listas dentro de listas: MutableList



En Kotlin, una matriz 2D es simplemente una MutableList donde cada elemento es otra MutableList. Su naturaleza "Mutable" permite actualizar el estado (de AGUA a BARCO) en tiempo real.

Creando el océano en una sola línea

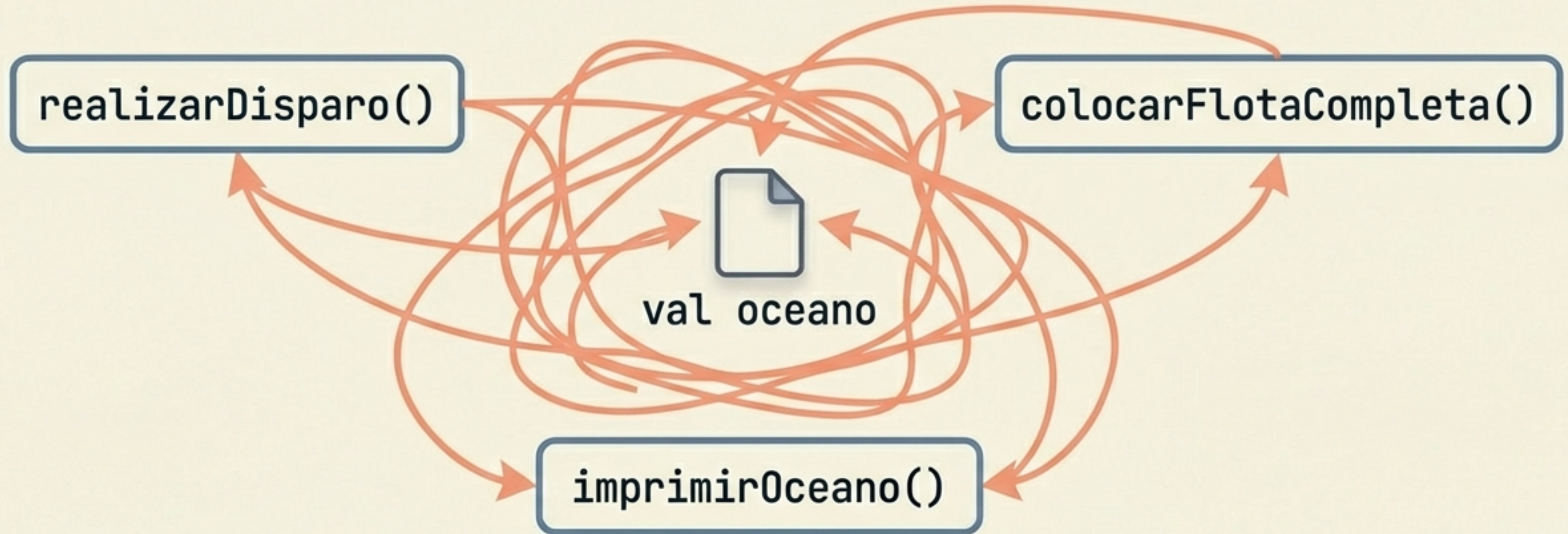
Crea 10 filas.

```
MutableList(DIMENSION) {  
    MutableList(DIMENSION) { EstadoCasilla.AGUA  
} }
```

Para cada fila, crea
10 columnas.

Rellena cada casilla con el estado
por defecto usando nuestro Enum.

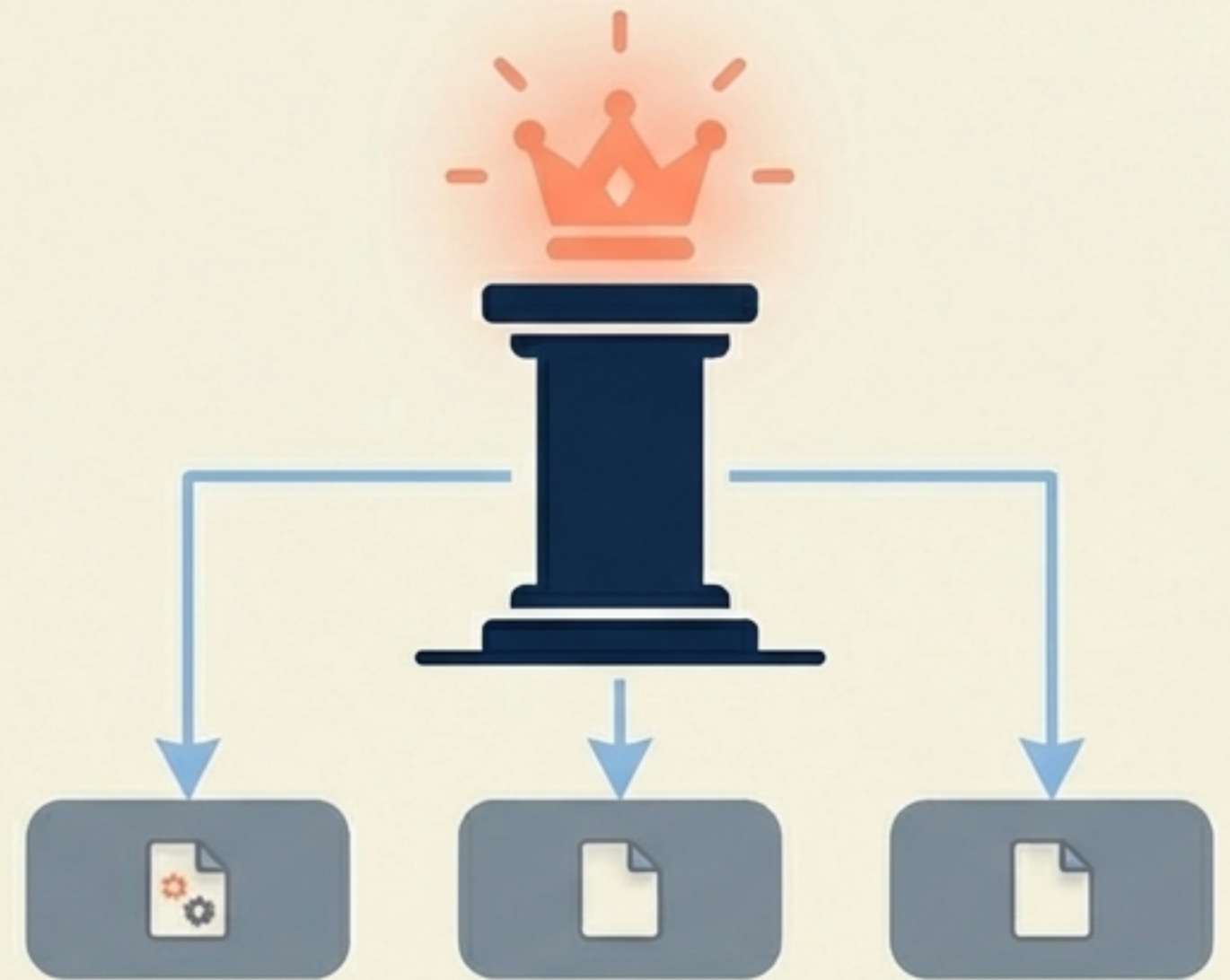
El riesgo de perder el estado del juego



Si pasamos la matriz del océano de función en función como un parámetro normal, el código se vuelve frágil y confuso. Necesitamos una única fuente de verdad.

Object: El director de orquesta (Singleton)

```
object Juego {  
  val oceano = MutableList  
    (DIMENSION) { ... }  
  val impactosNecesarios = 17  
}
```



La palabra clave `object` crea un Singleton. Garantiza que solo exista UNA única instancia de `Juego` en toda la memoria. Todas las funciones pueden leer y modificar el mismo **`Juego.oceano`** de forma segura.

Sinergia: Todo conectado en el océano

```
if (horizontal) {  
    Juego.oceano[f][c + i] = EstadoCasilla.BARCO  
}
```

Juego : Accedemos
al Singleton.

.oceano[f][c + i] :
Modificamos la
MutableList anidada.

= EstadoCasilla.BARCO
: Asignamos un valor
seguro del Enum.

Tres conceptos avanzados de Kotlin colaborando perfectamente
en una sola línea de código lógico.

Recorriendo estructuras sin esfuerzo

```
for (barco in TipoBarco.values())
```

Itera automáticamente por todos los barcos definidos sin crear arrays manuales.

```
for (c in Juego.oceano.indices)
```

Equivale matemáticamente a 0 until size. Previene errores de desbordamiento al dibujar el tablero.

Separando los datos de la lógica



-  **Enums** eliminan los errores tipográficos y empaquetan datos constantes.
-  **Objects** previenen el caos de estado manteniendo una sola "fuente de verdad".
-  **MutableLists** proveen la memoria dinámica necesaria para interactuar.

Esta arquitectura no es solo para Hundir la Flota. Es la base de un diseño de software escalable.

Tu turno de tomar el timón

```
--- HUNDIR LA FLOTA (Versión Listas) ---  
Tablero: 10x10  
Objetivo: 17 impactos.
```

Ya dominas las herramientas fundamentales. La mejor forma de asentar este conocimiento es romper el código y volver a armarlo.

- ¿Qué pasa si cambias `DIMENSION = 10` a `20`?
- ¡Intenta añadir un nuevo Enum para un `TipoBarco.LANCHA(1)`!